

JavaScript Intermediate — Classes & Inheritance

Section 15.3 — `class`, Constructor, Methods, Getters/Setters, `extends` & `super`

1. Class কী?

JavaScript-এর `class` হলো object তৈরি এবং inheritance organize করার একটি cleaner syntax ।

JavaScript আসলে **prototype-based language** । `class` ব্যবহার করলেও ভিতরে prototype system-ই কাজ করে ।

```
class User {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    return `Hello, I am ${this.name}`;  
  }  
}  
  
const user1 = new User("Shariar", 31);  
  
console.log(user1.name);  
console.log(user1.greet());
```

Output:

```
Shariar  
Hello, I am Shariar
```

এখানে:

```
class User
```

একটি class ।

```
constructor(name, age)
```

object তৈরি হওয়ার সময় automatically call হয়।

```
new User("Shariar", 31)
```

একটি নতুন instance তৈরি করে।

2. Class Syntax

Basic structure:

```
class ClassName {  
  constructor(parameters) {  
    // initialize instance data  
  }  
  
  methodName() {  
    // method  
  }  
}
```

Example:

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  
  sayHello() {  
    console.log(`Hello ${this.name}`);  
  }  
}
```

Create object:

```
const person = new Person("Duke");  
  
person.sayHello();
```

Output:

Hello Duke

3. constructor() Method

constructor() হলো special method ।

যখন `new` দিয়ে class থেকে object তৈরি করা হয়, তখন constructor automatically execute হয় ।

```
class User {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
}  
  
const user = new User("Duke", 31);
```

Conceptually:

```
new User(...)  
  ↓  
new object তৈরি  
  ↓  
constructor() execute  
  ↓  
this → নতুন object  
  ↓  
object return
```

4. this Inside Constructor

Constructor-এর ভিতরে `this` current instance-কে refer করে ।

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

```
    }  
  }  
  
  const user1 = new User("Shariar");  
  const user2 = new User("Rahim");  
  
  console.log(user1.name);  
  console.log(user2.name);
```

Output:

```
Shariar  
Rahim
```

এখানে:

```
this.name = name;
```

`user1` তৈরি হলে:

```
user1.name = "Shariar";
```

আর `user2` তৈরি হলে:

```
user2.name = "Rahim";
```

5. Class Methods

Class-এর ভিতরে methods define করা যায়।

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hello ${this.name}`;  
  }  
}
```

```
logout() {  
  return `${this.name} logged out`;  
}  
}
```

Use:

```
const user = new User("Duke");  
  
console.log(user.greet());  
console.log(user.logout());
```

6. Class Methods কোথায় থাকে?

এটা খুব important ।

Class-এর methods সাধারণত প্রতিটি object-এর ভিতরে আলাদা করে তৈরি হয় না।

```
class User {  
  greet() {  
    console.log("Hello");  
  }  
}
```

`greet()` থাকে:

`User.prototype`

এর মধ্যে ।

অর্থাৎ:

```
user  
↓  
User.prototype  
↓  
Object.prototype  
↓  
null
```

তাই:

```
const user1 = new User();
const user2 = new User();

console.log(user1.greet === user2.greet);
```

Output:

```
true
```

কারণ দুই instance একই prototype method ব্যবহার করছে।

এটাই আগের **prototype** concept-এর সাথে class-এর connection।

7. Class Instance বনাম Prototype

ধরো:

```
class User {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return `Hello ${this.name}`;
  }
}
```

তাহলে:

```
User
├── prototype
│   └── greet()
└── user1
    ├── name
    └──
```

```
└─ [[Prototype]]  
  ↓  
  User.prototype
```

Instance-specific data:

```
this.name
```

Shared behavior:

```
greet()
```

এটা memory-efficient design ।

8. Class Fields / Instance Fields

Modern JavaScript-এ constructor-এর বাইরে সরাসরি fields declare করা যায় ।

```
class User {  
  role = "user";  
  
  constructor(name) {  
    this.name = name;  
  }  
}
```

Use:

```
const user = new User("Duke");  
  
console.log(user.name);  
console.log(user.role);
```

Output:

```
Duke  
user
```

এগুলো **instance properties** ।

প্রতিটি instance নিজের field পায়।

```
const user1 = new User("Duke");
const user2 = new User("Rahim");

console.log(user1.role);
console.log(user2.role);
```

9. Constructor vs Class Fields

দুইভাবেই instance property তৈরি করা যায়।

Constructor

```
class User {
  constructor(name) {
    this.name = name;
  }
}
```

Class field

```
class User {
  name = "Unknown";
}
```

Dynamic value হলে constructor খুব common:

```
class User {
  constructor(name) {
    this.name = name;
  }
}
```

Default value হলে field convenient:

```
class User {  
  role = "user";  
}
```

দুটো একসাথেও ব্যবহার করা যায়:

```
class User {  
  role = "user";  
  
  constructor(name) {  
    this.name = name;  
  }  
}
```

10. Getters

Getter ব্যবহার করে property-এর মতো syntax-এ method-এর result access করা যায়।

```
class User {  
  constructor(firstName, lastName) {  
    this.firstName = firstName;  
    this.lastName = lastName;  
  }  
  
  get fullName() {  
    return `${this.firstName} ${this.lastName}`;  
  }  
}
```

Use:

```
const user = new User("Shariar", "Duke");  
  
console.log(user.fullName);
```

Output:

```
Shariar Duke
```

খোঁজা করা:

```
user.fullName
```

এখানে:

```
user.fullName()
```

লিখতে হয় না।

কারণ `fullName` getter।

11. Getter কেন ব্যবহার করব?

Getter useful যখন কোনো property অন্য properties থেকে calculate করা হয়।

Example:

```
class Rectangle {  
  constructor(width, height) {  
    this.width = width;  
    this.height = height;  
  }  
  
  get area() {  
    return this.width * this.height;  
  }  
}  
  
const rectangle = new Rectangle(10, 5);  
  
console.log(rectangle.area);
```

Output:

```
50
```

এখানে `area` stored property না।

প্রয়োজনের সময় calculate হচ্ছে।

12. Setters

Setter ব্যবহার করে কোনো property assign করার সময় custom logic চালানো যায়।

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  set username(value) {  
    this.name = value.trim();  
  }  
}
```

Use:

```
const user = new User("Duke");  
  
user.username = " Shariar ";  
  
console.log(user.name);
```

Output:

```
Shariar
```

এখানে:

```
user.username = " Shariar ";
```

করলে setter execute হয়েছে।

13. Getter + Setter Together

```
class User {  
  constructor(name) {  
    this._name = name;  
  }  
}
```

```
get name() {  
    return this._name;  
}  
  
set name(value) {  
    this._name = value.trim();  
}  
}
```

Use:

```
const user = new User("Duke");  
  
user.name = " Sharar ";  
  
console.log(user.name);
```

Output:

Sharar

এখানে:

_name

একটি convention হিসেবে internal storage বোঝাতে ব্যবহার করা হয়েছে।

এটা নিজে থেকে private নয়।

Private field-এর জন্য JavaScript-এ:

#name

ব্যবহার করা হয়।

এটা পরের advanced section-এ বিস্তারিত আসবে।

14. Getter/Setter-এর Common Mistake

এভাবে লিখলে সমস্যা হবে:

```
class User {  
  get name() {  
    return this.name;  
  }  
}
```

কারণ:

```
this.name
```

আবার getter-কে call করবে।

ফলে infinite recursion হবে।

সাধারণ pattern:

```
class User {  
  constructor(name) {  
    this._name = name;  
  }  
  
  get name() {  
    return this._name;  
  }  
  
  set name(value) {  
    this._name = value;  
  }  
}
```

15. Class Expression

Class declaration ছাড়াও class expression করা যায়।

```
const User = class {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return `Hello ${this.name}`;
  }
};
```

তারপর:

```
const user = new User("Duke");

console.log(user.greet());
```

সাধারণ application code-এ class declaration বেশি readable:

```
class User {
  // ...
}
```

16. Named Class Expression

Class expression-এর নিজের নামও থাকতে পারে।

```
const User = class UserClass {
  constructor(name) {
    this.name = name;
  }
};
```

বাইরে variable:

```
User
```

কিন্তু class-এর internal name:

```
UserClass
```

এটা advanced use case ।

17. Class Hoisting

Function declaration-এর মতো class declaration সাধারণভাবে hoist হয়ে usable হয় না ।

এটা ভুল:

```
const user = new User();

class User {
  constructor() {}
}
```

এখানে error হবে ।

Class declaration lexical scope-এর মধ্যে থাকে এবং initialization-এর আগে access করা যায় না ।

এটা `let` / `const` -এর TDZ-এর সাথে conceptually related ।

সঠিক:

```
class User {
  constructor() {}
}

const user = new User();
```

18. Classes Are Strict Mode

Class body automatically strict mode-এর মতো আচরণ করে ।

```
class User {
  method() {
    // strict mode behavior
  }
}
```

```
}  
}
```

তাই class-এর ভিতরে JavaScript-এর strict rules apply হয়।

আলাদা করে:

```
"use strict";
```

দেওয়ার প্রয়োজন হয় না class body strict করার জন্য।

19. extends — Inheritance

একটি class অন্য class থেকে properties এবং methods inherit করতে পারে।

Syntax:

```
class Child extends Parent {  
}
```

Example:

```
class User {  
  greet() {  
    return "Hello";  
  }  
}  
  
class Admin extends User {  
  deleteUser() {  
    return "User deleted";  
  }  
}
```

এখন:

```
const admin = new Admin();
```

```
console.log(admin.greet());  
console.log(admin.deleteUser());
```

Output:

```
Hello  
User deleted
```

`Admin` নিজে `greet()` define করেনি।

এটা `User` থেকে পেয়েছে।

20. Inheritance Structure

উপরের example-এর prototype chain:

```
admin  
  ↓  
Admin.prototype  
  ↓  
User.prototype  
  ↓  
Object.prototype  
  ↓  
null
```

এটাই prototype-based inheritance।

`class extends` শুধু এই inheritance relationship লেখার cleaner syntax দেয়।

21. Parent Class

যে class থেকে inheritance করা হয় তাকে সাধারণভাবে:

Parent / Base / Superclass

বলা হয়।

```
class User {  
}
```

এখানে `User` parent class হতে পারে।

22. Child Class

যে class inherit করে:

Child / Derived / Subclass

```
class Admin extends User {  
}
```

এখানে:

```
User → Parent  
Admin → Child
```

23. `super()`

Child class-এর constructor-এ parent constructor call করতে:

```
super()
```

ব্যবহার করতে হয়।

Example:

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
class Admin extends User {  
  constructor(name, role) {
```

```
    super(name);  
    this.role = role;  
  }  
}
```

Use:

```
const admin = new Admin("Duke", "admin");  
  
console.log(admin.name);  
console.log(admin.role);
```

Output:

```
Duke  
admin
```

24. কেন `super()` দরকার?

Derived class-এর constructor-এ `this` ব্যবহার করার আগে parent constructor initialize করতে হয়।

ভুল:

```
class Admin extends User {  
  constructor(name, role) {  
    this.role = role;  
    super(name);  
  }  
}
```

এটা error করবে।

সঠিক:

```
class Admin extends User {  
  constructor(name, role) {  
    super(name);  
    this.role = role;  
  }  
}
```

```
}  
}
```

Mental model:

```
Admin constructor  
  ↓  
super(name)  
  ↓  
User constructor  
  ↓  
this.name তৈরি  
  ↓  
Admin-এর own properties
```

25. Child Class-এ Constructor না লিখলে?

Child class যদি নিজের constructor define না করে, তাহলে inherited constructor behavior পাওয়া যায়।

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
class Admin extends User {  
}
```

তখন:

```
const admin = new Admin("Duke");  
  
console.log(admin.name);
```

Output:

Duke

26. `super.method()`

`super()` শুধু constructor-এর জন্য না।

Parent class-এর method call করতেও:

```
super.method()
```

ব্যবহার করা যায়।

Example:

```
class User {  
  greet() {  
    return "Hello from User";  
  }  
}  
  
class Admin extends User {  
  greet() {  
    return `${super.greet()} and Admin`;  
  }  
}
```

Use:

```
const admin = new Admin();  
  
console.log(admin.greet());
```

Output:

```
Hello from User and Admin
```

এখানে child method parent method-এর functionality extend করেছে।

27. Method Overriding

Child class parent-এর method নিজের মতো করে redefine করতে পারে।

```
class User {
  greet() {
    return "Hello User";
  }
}

class Admin extends User {
  greet() {
    return "Hello Admin";
  }
}
```

এখন:

```
const user = new User();
const admin = new Admin();

console.log(user.greet());
console.log(admin.greet());
```

Output:

```
Hello User
Hello Admin
```

এটাকে **method overriding** বলে।

28. Override + super

Parent-এর functionality রেখে additional functionality যোগ করতে:

```
class User {
  greet() {
    return "Hello";
  }
}

class Admin extends User {
  greet() {
    return `${super.greet()} Admin`;
  }
}
```

```
}  
}
```

এটা খুব useful pattern ।

29. Inherited Properties

Parent constructor-এর মাধ্যমে তৈরি property-ও child instance পেতে পারে ।

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
class Admin extends User {  
  constructor(name, role) {  
    super(name);  
    this.role = role;  
  }  
}
```

তাহলে:

```
const admin = new Admin("Duke", "admin");  
  
console.log(admin.name);  
console.log(admin.role);
```

Output:

```
Duke  
admin
```

30. Multi-Level Inheritance

একাধিক level inheritance সম্ভব ।

```

class Animal {
  eat() {
    return "Eating";
  }
}

class Dog extends Animal {
  bark() {
    return "Barking";
  }
}

class Puppy extends Dog {
  play() {
    return "Playing";
  }
}

```

Prototype relationship:

```

puppy
↓
Puppy.prototype
↓
Dog.prototype
↓
Animal.prototype
↓
Object.prototype
↓
null

```

তাই:

```

const puppy = new Puppy();

puppy.eat();
puppy.bark();
puppy.play();

```

তিনটিই কাজ করবে।

31. `extends` শুধু **Class**-এর সাথেই নয়

JavaScript-এর inheritance prototype-based হওয়ায় `extends` বিভিন্ন constructor/prototype-based setup-এর সাথেও কাজ করতে পারে।

তবে practical development-এ সাধারণত:

```
class Child extends Parent
```

pattern-টাই বেশি readable।

32. Class vs Constructor Function

আগের section-এ আমরা constructor function দেখেছি।

পুরনো style:

```
function User(name) {  
  this.name = name;  
}  
  
User.prototype.greet = function () {  
  return `Hello ${this.name}`;  
};
```

Modern class syntax:

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hello ${this.name}`;  
  }  
}
```

দুটোর underlying prototype concept related।

Class syntax মূলত prototype-based behavior লেখাকে cleaner করে।

33. Class Method বনাম Instance Method

Class-এর সাধারণ method:

```
class User {  
  greet() {  
    return "Hello";  
  }  
}
```

এটা instance-এর মাধ্যমে access করা হয়:

```
const user = new User();  
  
user.greet();
```

এগুলোকে সাধারণভাবে instance methods বলা হয়।

`static` methods instance-এর মাধ্যমে access করা হয় না।

```
User.someMethod();
```

Static methods পরের section-এ বিস্তারিত।

34. Getter/Setter Practical Example

ধরো bank account:

```
class BankAccount {  
  constructor(balance) {  
    this._balance = balance;  
  }  
  
  get balance() {  
    return this._balance;  
  }  
  
  set balance(value) {  
    if (value < 0) {
```

```
        throw new Error("Balance cannot be negative");
    }

    this._balance = value;
}
}
```

Use:

```
const account = new BankAccount(1000);

console.log(account.balance);

account.balance = 2000;

console.log(account.balance);
```

Output:

```
1000
2000
```

Invalid value:

```
account.balance = -500;
```

এখানে error throw হবে।

Getter/setter validation-এর জন্য useful।

35. Real-World Example — User/Admin

```
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }

  login() {
    return `${this.name} logged in`;
  }
}
```

```

    }
}

class Admin extends User {
  constructor(name, email, permissions) {
    super(name, email);
    this.permissions = permissions;
  }

  deleteUser() {
    return "User deleted";
  }
}

```

Use:

```

const admin = new Admin(
  "Shariar",
  "shariar@example.com",
  ["delete", "update"]
);

console.log(admin.name);
console.log(admin.email);
console.log(admin.login());
console.log(admin.deleteUser());

```

এখানে Admin:

- নিজের `permissions` রাখে
- নিজের `deleteUser()` রাখে
- parent থেকে `name` পায়
- parent থেকে `email` পায়
- parent থেকে `login()` পায়

36. Important Mental Model

Class দেখে মনে হতে পারে:

```
Class
↓
Objects
```

কিন্তু JavaScript-এর deeper mental model:

```
Class syntax
↓
Prototype system
↓
Prototype chain
↓
Property lookup
```

Inheritance:

```
Child instance
↓
Child.prototype
↓
Parent.prototype
↓
Object.prototype
↓
null
```

37. Property Lookup কীভাবে হয়?

ধরো:

```
class User {
  greet() {
    return "Hello";
  }
}

class Admin extends User {}

const admin = new Admin();
```

```
admin.greet();
```

JavaScript প্রথমে খুঁজবে:

```
admin
```

এখানে `greet` নেই।

তারপর:

```
Admin.prototype
```

এখানেও নেই।

তারপর:

```
User.prototype
```

এখানে `greet` পাওয়া গেল।

তাই method execute হবে।

38. `instanceof` Preview

Class inheritance-এর সাথে `instanceof` খুব closely related।

```
class User {}

class Admin extends User {}

const admin = new Admin();

console.log(admin instanceof Admin);
console.log(admin instanceof User);
console.log(admin instanceof Object);
```

Output:

```
true
true
true
```

কারণ prototype chain-এর মধ্যে:

```
Admin.prototype
User.prototype
Object.prototype
```

আছে।

`instanceof` পরের section-এ বিস্তারিতভাবে আসবে।

39. Class-এর ভিতরে Arrow Function Field

Class field হিসেবে arrow function-ও ব্যবহার করা যায়।

```
class User {
  name = "Duke";

  greet = () => {
    return `Hello ${this.name}`;
  };
}
```

এখানে `greet` প্রতিটি instance-এর নিজস্ব property হিসেবে তৈরি হবে।

এটা prototype method-এর মতো shared নয়।

এই difference performance এবং `this` behavior-এর ক্ষেত্রে গুরুত্বপূর্ণ।

সাধারণভাবে:

```
class User {
  greet() {
    // prototype method
  }
}
```

এবং:

```
class User {  
  greet = () => {  
    // instance field  
  };  
}
```

এক জিনিস নয়।

40. Prototype Method vs Arrow Function Field

Prototype method

```
class User {  
  greet() {  
    return "Hello";  
  }  
}
```

একই prototype method share করে।

Arrow field

```
class User {  
  greet = () => {  
    return "Hello";  
  };  
}
```

প্রতিটি instance নিজের `greet` function পায়।

Arrow field-এর সুবিধা হলো lexical `this` behavior, বিশেষ করে callback/event handler-এর ক্ষেত্রে।

তবে অপ্রয়োজনে সব method arrow field বানানো উচিত নয়।

41. `super` এবং Method Overriding — Full Example

```
class Employee {
  constructor(name) {
    this.name = name;
  }

  getRole() {
    return "Employee";
  }

  introduce() {
    return `${this.name} is an ${this.getRole()}`;
  }
}

class Developer extends Employee {
  getRole() {
    return "Developer";
  }
}
```

Use:

```
const developer = new Developer("Duke");

console.log(developer.introduce());
```

Output:

```
Duke is an Developer
```

এখানে `introduce()` parent class-এর method ।

কিন্তু parent method-এর ভিতরের:

```
this.getRole()
```

child-এর overridden method call করছে ।

এটা polymorphic behavior-এর একটি basic example ।

42. Common Mistakes

Mistake 1 — `new` ছাড়া `class call` করা

```
class User {}  
  
const user = User();
```

❌ Error ।

সঠিক:

```
const user = new User();
```

Mistake 2 — Child constructor-এ `super()` না দেওয়া

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
class Admin extends User {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

❌ Derived constructor-এ `super()` ছাড়া `this` ব্যবহার করা যাবে না ।

সঠিক:

```
class Admin extends User {  
  constructor(name) {  
    super(name);  
  }  
}
```

Mistake 3 — Getter-কে function হিসেবে call করা

```
class User {  
  get name() {  
    return "Duke";  
  }  
}  
  
const user = new User();  
  
user.name();
```

❌ ভুল।

সঠিক:

```
user.name;
```

Mistake 4 — Setter-কে manually call করা

```
user.name("Duke");
```

Setter-এর ক্ষেত্রে assignment syntax:

```
user.name = "Duke";
```

Mistake 5 — `super()` এর আগে `this`

ভুল:

```
constructor(name) {  
  this.name = name;  
  super(name);  
}
```

সঠিক:

```
constructor(name) {  
  super(name);  
  this.role = "admin";  
}
```

43. Output Prediction

Example 1

```
class User {  
  greet() {  
    return "User";  
  }  
}  
  
class Admin extends User {  
  greet() {  
    return "Admin";  
  }  
}  
  
const admin = new Admin();  
  
console.log(admin.greet());
```

Output:

```
Admin
```

কারণ child method parent method override করেছে।

Example 2

```
class User {  
  greet() {  
    return "Hello";  
  }  
}  
  
class Admin extends User {
```

```
    greet() {  
        return super.greet() + " Admin";  
    }  
}  
  
const admin = new Admin();  
  
console.log(admin.greet());
```

Output:

```
Hello Admin
```

Example 3

```
class User {  
    constructor(name) {  
        this.name = name;  
    }  
}  
  
class Admin extends User {  
    constructor(name) {  
        super(name);  
        this.role = "admin";  
    }  
}  
  
const admin = new Admin("Duke");  
  
console.log(admin.name);  
console.log(admin.role);
```

Output:

```
Duke  
admin
```

44. Practice Tasks

Task 1 — Person

একটি `Person` class বানাও।

Properties:

```
name  
age
```

Method:

```
introduce()
```

Output:

```
My name is Duke and I am 31 years old.
```

Task 2 — Student

`Person` থেকে `Student` inherit করাও।

Student-এর extra property:

```
course
```

Method:

```
study()
```

Task 3 — Getter

`Rectangle` class বানাও:

```
width  
height
```

Getter:

```
area
```

যাতে:

```
rectangle.area
```

দিলে area পাওয়া যায়।

Task 4 — Setter Validation

`User` class বানাও।

Property:

```
age
```

Setter-এর মাধ্যমে নিশ্চিত করে:

```
age >= 0
```

না হলে error throw করবে।

Task 5 — Method Overriding

Classes:

```
Animal  
Dog  
Cat
```

`Animal`:

```
 speak()
```

Dog এবং Cat নিজেদের মধ্যে `speak()` override করবে।

45. Interview Questions

Beginner

1. JavaScript class কী?
2. `constructor()` কী?
3. `new` কেন ব্যবহার করা হয়?
4. class method কোথায় থাকে?
5. `extends` কী?
6. `super()` কী?
7. getter এবং setter কী?
8. method overriding কী?

Intermediate

1. JavaScript কি class-based নাকি prototype-based?
2. Class এবং constructor function-এর relationship কী?
3. Class methods কি প্রতিটি object-এর ভিতরে duplicate হয়?
4. `super()` কেন `this`-এর আগে call করতে হয়?
5. `super.method()` কী করে?
6. class declaration কি hoist হয়?
7. class field এবং prototype method-এর difference কী?
8. `extends` prototype chain কীভাবে তৈরি করে?

Advanced

1. `class` syntax-এর নিচে কী ধরনের inheritance system কাজ করে?
2. `super` কীভাবে parent prototype-এর method access করে?
3. `instanceof` এবং inheritance-এর relationship কী?
4. Prototype method বনাম arrow-function class field-এর trade-off কী?

46. Quick Reference

Feature	Purpose
<code>class</code>	Object blueprint তৈরি
<code>constructor()</code>	Instance initialize

Feature	Purpose
<code>this</code>	Current instance
<code>method()</code>	Instance behavior
<code>get</code>	Property-like computed access
<code>set</code>	Property assignment control
<code>extends</code>	Inheritance
<code>super()</code>	Parent constructor call
<code>super.method()</code>	Parent method call
class field	Instance property
method overriding	Parent method replace/customize
<code>instanceof</code>	Prototype-chain based instance check

47. Final Mental Model

JavaScript class শেখার সবচেয়ে important flow:

```

class User
  ↓
new User()
  ↓
new instance
  ↓
instance properties
  ↓
User.prototype
  ↓
shared methods

```

Inheritance হলে:

```

Admin instance
  ↓
Admin.prototype
  ↓
User.prototype
  ↓

```

```
Object.prototype  
  ↓  
null
```

আর:

```
extends
```

এই prototype relationship তৈরি করার cleaner syntax ।

```
super()
```

parent constructor-এর functionality ব্যবহার করে ।

```
super.method()
```

parent method-এর functionality ব্যবহার করে ।

48. Mastery Checklist

এই section শেষ করার পর তুমি explain করতে পারবে:

- [] `class` কী
- [] `constructor()` কীভাবে কাজ করে
- [] `this` কী refer করে
- [] class methods কোথায় থাকে
- [] instance fields কী
- [] getter কী
- [] setter কী
- [] getter/setter validation
- [] class expression
- [] class hoisting/TDZ behavior
- [] `extends`
- [] parent/child class
- [] `super()`
- [] `super.method()`
- [] method overriding
- [] multi-level inheritance
- [] class বনাম constructor function
- [] prototype chain-এর সাথে class-এর relationship
- [] prototype method বনাম instance arrow-function field

- [] instanceof কীভাবে inheritance-এর সাথে related
-

One-Line Summary

JavaScript `class` হলো **prototype-based object system**-এর উপর তৈরি **cleaner syntax**; `extends` **inheritance** তৈরি করে, `super()` **parent functionality access** করে, আর **class methods** সাধারণত **prototype**-এ **shared** থাকে।